Relatório de Auditoria de Segurança

ddlab-data

21 de outubro de 2026 (2ª rodada — atualização)

Escopo auditado

Repositório **dfassuncao/ddlab-data** (branch `main`, HEAD 06d80a4): aplicação interna da agência DDLab para inteligência de campanhas Google Ads/GA4/Search Console — Cloudflare Worker (Hono) como backend/API, SPA React como frontend, Cloudflare D1 (SQLite) como banco pré-agregado, BigQuery como fonte de dados, Cloudflare Access como camada de login, e uma integração de escrita com a Google Ads API (negativar termos, pausar/reativar campanhas e palavras-chave, Fases 1 e 2, commits #36-#38) adicionada desde a 1ª rodada desta auditoria. Todos os arquivos de `worker/` (incluindo as rotas novas `routes/adsActions.ts` e `googleAds.ts`), `src/`, `shared/`, `migrations/` e arquivos de configuração/deploy (wrangler.toml, DEPLOY.md, .dev.vars.example, package.json, scripts/deploy.sh) foram revisados; o histórico completo do git foi varrido novamente em busca de segredos commitados. Esta é a **2ª rodada** da auditoria — os achados F1 e F4 da rodada anterior (fail-open de autenticação e injeção de fórmula em CSV) foram verificados como corrigidos e migraram para a seção de pontos fortes.

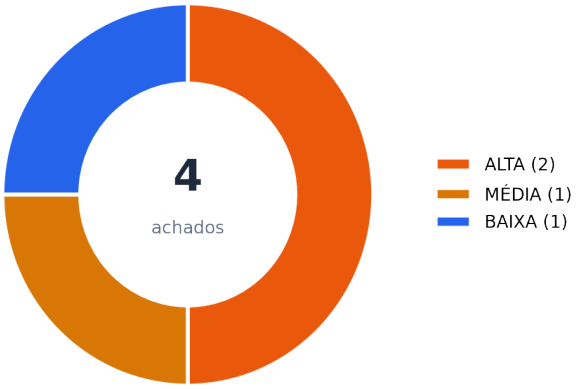
Nota metodológica — mapeamento das 5 categorias para esta stack

1. Isolamento de inquilino/dono: a aplicação é single-tenant por desenho (uma agência, múltiplas contas de clientes finais, sem conceito de usuário-por-cliente) — o mecanismo de "tranca" é o Cloudflare Access na borda mais a verificação de JWT em worker/auth.ts. Auditado como "esse mecanismo pode falhar aberto?" e "há segmentação de dados por conta dentro do banco?".
2. Permissão definida no navegador: mapeado como "o frontend esconde alguma ação atrás de papel/isAdmin e o backend replica essa checagem?" — buscado em todo `src/` por lógica de papel/permissão.
3. IDOR: todos os handlers de rota de `worker/routes/*.ts` e `worker/index.ts` foram percorridos individualmente, verificando se buscas/updates por `?account=` ou `id` no corpo confirmam posse.
4. Chaves expostas: checado wrangler.toml, .dev.vars.example, package.json, migrations, seed/,

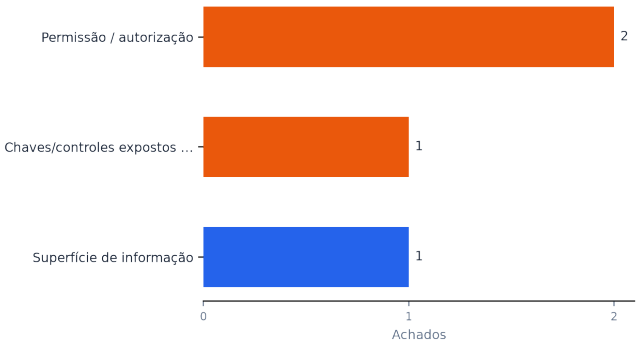
DEPLOY.md/README.md e histórico do git (`git log --all -p`) por segredo real, e cada var não-secreta com default vazio por ausência de validação de startup.
5. Inputs sem tratamento (XSS): mapeado para React (`dangerouslySetInnerHTML`, renderização de Markdown) no frontend, e para escaping em exportação CSV/templates no backend.

Resumo executivo

Severidade	Qtde
ALTA	2
MÉDIA	1
BAIXA	1



Achados por severidade



Achados por categoria

A auditoria percorreu 100% dos handlers de rota do backend (worker/index.ts e worker/routes/*.ts, incluindo a rota nova de ações no Google Ads), o middleware de autenticação, a camada de acesso a dados (D1 e BigQuery), o ETL, a integração de escrita com a Google Ads API e os pontos de renderização do frontend. Foram identificados **4 achados** nesta rodada — destaque para **F1 (alta)**: a ausência de RBAC agora também cobre mutações reais em contas de clientes no Google Ads, não só leitura de dados. Os dois achados críticos/médios da rodada anterior (autenticação fail-open e injeção de fórmula em CSV) foram verificados como corrigidos. Como pontos fortes, o código continua evitando consistentemente SQL injection (D1 e BigQuery) e XSS via HTML bruto, a nova integração de escrita valida entrada por whitelist, e nenhum segredo real foi encontrado no histórico do git.

Pontos fortes

O que já está protegido, com a evidência verificada no código.

PONTO FORTE **RESOLVIDO desde a 1ª rodada — autenticação do Cloudflare Access não falha mais aberta quando a configuração está ausente**

O achado crítico F1 da auditoria anterior (fail-open quando CF_ACCESS_TEAM_DOMAIN/AUD estavam vazios e commitados em [vars]) está corrigido: wrangler.toml não declara mais essas duas chaves em [vars] (comentário explícito nas linhas 32-40 do porquê), e worker/auth.ts:97-104 agora retorna 500 fail-closed sempre que `c.env.ENVIRONMENT === "production"` e as vars estiverem ausentes — o fallback `dev@localhost` só é possível fora de produção. DEPLOY.md documenta a decisão (linhas 111-151) e scripts/deploy.sh reaplica os dois secrets a cada deploy para evitar regressão.

PONTO FORTE **RESOLVIDO desde a 1ª rodada — exportação CSV neutraliza injeção de fórmula (CSV/Excel formula injection)**

O achado F4 da auditoria anterior está corrigido: src/lib/csv.ts:1-6 agora tem um `csvField()` central que prefixa com apóstrofo qualquer valor iniciando com `=`, `+`, `-` ou `@` antes de montar a linha CSV, e tanto `downloadNegativesCsv` (ReportPage.tsx) quanto `exportCsv` (TermClassification.tsx) foram migrados para usar o `downloadCsv()` compartilhado — não há mais um caminho de exportação que monte a linha CSV manualmente.

PONTO FORTE **Todas as queries ao D1 usam parâmetros bind (?), sem concatenação de entrada do usuário em SQL**

Verificado em worker/routes/reports.ts, analysis.ts, decision.ts, adsActions.ts e rules.ts — o helper `q(env, sql, binds)` sempre usa `.bind(...binds)`. Os únicos identificadores de tabela/coluna interpolados diretamente no SQL vêm de mapas estáticos no código (`KINDS` em reports.ts, `FACTS` em etl/queries.ts), nunca de query string ou body do usuário.

PONTO FORTE **Consultas ao BigQuery neutralizam identificadores controláveis pelo usuário antes de interpolar no SQL**

worker/etl/ga4.ts:5,90-92 valida `ga4_dataset` e cada item de `ga4_key_events` (campos editáveis em Configurações) com o regex `^[a-zA-Z0-9_]+$` (safeIdent) antes de usá-los na montagem da query GA4; mesmo padrão em worker/etl/gsc.ts:5,59 para `gsc_dataset`. `customer_id` é passado como parâmetro nomeado do BigQuery, não concatenado.

PONTO FORTE **Segredos reais nunca foram commitados no repositório (confirmado novamente nesta rodada)**

.gitignore exclui .dev.vars, service-account*.json, *.key.json e o padrão específico do arquivo baixado do GCP (studio-*-*.json, *-de430-*.json). `git log --all -p | grep -i "PRIVATE KEY|AIzaSy|sk-ant|GCP_SA_KEY="` não retornou nenhuma chave privada, API key ou token real em nenhum commit do histórico — apenas o placeholder "..." em .dev.vars.example. Segredos (GCP_SA_KEY, ANTHROPIC_API_KEY, GEMINI_API_KEY, DEEPSEEK_API_KEY, GOOGLE_ADS_*, CF_ACCESS_*) são todos aplicados via `wrangler secret put`, fora do controle de versão — nenhum deles aparece em wrangler.toml [vars].

PONTO FORTE **Verificação de JWT do Cloudflare Access implementada corretamente quando configurada**

worker/auth.ts:60-87 valida assinatura RS256 contra o JWKS público do Access (cache de 1h), expiração (`exp`) e audience (`aud`, aceitando array ou string) antes de aceitar o token — nenhuma dessas checagens é pulada quando as vars estão presentes.

PONTO FORTE **Nenhum XSS de HTML bruto no frontend: conteúdo Markdown gerado por IA é renderizado com sanitização por padrão**

src/pages/AiAnalysis.tsx usa `` sem o plugin `rehype-raw` — por padrão o react-markdown NÃO renderiza tags HTML embutidas no texto, apenas Markdown. Busca em todo `src/` por `dangerouslySetInnerHTML`/`innerHTML`/`eval`/`new Function` não encontrou nenhum ponto de renderização de HTML não confiável.

PONTO FORTE **Integração de escrita com o Google Ads valida entrada com whitelist antes de montar a mutação, e o payload da mutação é gerado no servidor, não recebido do cliente**

worker/routes/adsActions.ts valida `matchType` contra ["BROAD","PHRASE","EXACT"] e `status` contra PAUSED/ENABLED antes de persistir a proposta; o `customerId` usado na mutação vem de `getAccount()` (nunca do body da requisição), e o payload gravado em `google_ads_actions` no momento do /propose-* é o mesmo lido e enviado no /approve — o cliente não pode alterar o alvo da mutação entre propor e aprovar.

PONTO FORTE **Geração de JWT para a service account do GCP e chamadas OAuth2 do Google Ads usam Web Crypto nativa, sem expor chave/token em logs**

worker/bq.ts:30-78 assina o JWT com `crypto.subtle.sign` e cacheia apenas o access token (não a chave privada) em memória do isolate, com expiração controlada; worker/googleAds.ts:9-30 segue o mesmo padrão de cache de token para o OAuth2 do

Google Ads.

Pontos fracos (riscos centrais)

O risco central é estrutural: não há segunda camada de autorização além do Cloudflare Access — todo usuário autenticado é, na prática, administrador de todas as contas de todos os clientes (F1), incluindo a capacidade de aprovar sozinho mutações reais no Google Ads que ele mesmo propôs (F2) e de gerar custo real em serviços pagos de terceiros sem controle adicional (F3). O mecanismo de autenticação em si (fail-open) e a exportação CSV (injeção de fórmula) — os dois riscos mais graves da rodada anterior — já foram corrigidos.

Achados detalhados

Permissão / autorização

Sev.	Arquivo:linha	Descrição
ALTA	worker/routes/reports.ts:753-803 worker/routes/adsActions.ts:60-261 worker/routes/analysis.ts:375-511 worker/index.ts:50-64	**Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes, incluindo mutações reais no Google Ads** Não existe conceito de papel (admin/editor/leitor) nem de dono de conta em nenhuma camada do backend (confirmado: nenhuma ocorrência de role/isAdmin/canEdit em worker/ ou src/). accessMiddleware() só verifica "é um e-mail permitido pela policy do Cloudflare Access" (DEPLOY.md linha 91: `@ddlab.com.br` ou e-mails individuais) — a partir daí, o mesmo usuário pode ler dados de todas as contas, editar metas/orçamento/briefing (POST /api/settings/account), disparar ETL pago para qualquer conta (POST /api/refresh), gerar análises de IA pagas (POST /api/analysis, /api/diagnostico) e — desde a integração de escrita com o Google Ads (Fases 1 e 2) — propor E aprovar mutações reais em qualquer conta: negativar termos, pausar/reativar campanhas e palavras-chave (worker/routes/adsActions.ts). Não há checagem de posse — é tudo-ou-nada.
MÉDIA	worker/routes/adsActions.ts:197-245	**Fluxo de aprovação de mutações no Google Ads não exige segregação entre quem propõe e quem aprova (sem maker-checker)** POST /api/ads-actions/:id/approve só verifica se a ação ainda está `pending` — não compara `action.requested_by` com `c.get("user").email`. O mesmo usuário que cria a proposta (negativar termo, pausar campanha/palavra-chave) pode aprová-la sozinho e disparar a mutação real no Google Ads (`addCampaignNegativeKeywords`/`setCampaignStatus`/`setKeywordStatus` em worker/googleAds.ts) sem qualquer segunda pessoa revisar. A fila de aprovação existe para dar visibilidade humana antes da mutação, mas nada no backend impede que essa revisão seja feita pela mesma pessoa que propôs.

Chaves/controles expostos ou inertes

Sev.	Arquivo:linha	Descrição
ALTA	worker/index.ts:50-64 .dev.vars.example:15-16	**Endpoint POST /api/refresh dispara custo real (BigQuery) sem limite de taxa; a proteção REFRESH_TOKEN documentada nunca é verificada no código** .dev.vars.example documenta REFRESH_TOKEN como "Token simples para proteger POST /api/refresh quando chamado fora do Access", e Env.REFRESH_TOKEN existe em worker/env.ts:19. Porém nenhum arquivo do worker lê ou compara REFRESH_TOKEN — o handler `app.post("/api/refresh", ...)` em worker/index.ts:50 dispara `runEtl()` (que consulta o BigQuery, custo real por bytes processados) sem checar esse token nem qualquer outro controle além do accessMiddleware() genérico, e sem rate limiting. Sem mudança desde a 1ª rodada da auditoria.

Superfície de informação

Sev.	Arquivo:linha	Descrição
BAIXA	worker/index.ts:23-39	**GET /api/introspect expõe o schema interno do BigQuery (nomes de coluna/tabela) para qualquer usuário autenticado** O endpoint de depuração /api/introspect roda `SELECT column_name, data_type FROM INFORMATION_SCHEMA.COLUMNS` para a tabela informada via querystring e devolve o schema. Está corretamente sob accessMiddleware() e o parâmetro `table` é validado por regex (`^[a-zA-Z0-9_]+$`) antes de entrar na query (sem SQL injection), mas é uma rota de debug interno deixada acessível em produção para qualquer usuário do Access, não só para quem ajusta o ETL. Sem mudança desde a 1ª rodada da auditoria.

Evidência técnica por achado

ALTA

F1 · Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes, incluindo mutações reais no Google Ads

`worker/routes/reports.ts:753-803, worker/routes/adsActions.ts:60-261, worker/routes/analysis.ts:375-511, worker/index.ts:50-64`

```
reports.post("/settings/account", async (c) => {
const body = await c.req.json<{ id: string; ... }>();
if (!body.id) return c.json({ error: "id obrigatório" }, 400);
await c.env.DB.prepare(`UPDATE dim_account SET ... WHERE id = ?`)...
// nenhuma verificação de quem pode editar a conta `body.id`
});

adsActions.post("/ads-actions/:id/approve", async (c) => {
// qualquer usuário autenticado aprova a mutação de QUALQUER conta
await addCampaignNegativeKeywords(c.env, payload.customerId, ...);
});
```

Condições de explorabilidade: Sempre explorável por qualquer conta liberada na policy do Cloudflare Access. Era um risco de exposição de dados/custo na 1ª rodada da auditoria; com a integração de escrita do Google Ads (commits #36-#38) o mesmo modelo tudo-ou-nada agora também cobre ações com efeito direto em campanhas reais de clientes (pausar campanha, negativar termo), o que amplia o dano possível de um acesso indevido ou de um colaborador despreparado. Ainda não documentado como decisão de produto deliberada em lugar nenhum do repositório.

MÉDIA

F2 · Fluxo de aprovação de mutações no Google Ads não exige segregação entre quem propõe e quem aprova (sem maker-checker)

`worker/routes/adsActions.ts:197-245`

```
adsActions.post("/ads-actions/:id/approve", async (c) => {
const action = await loadPending(c.env, id);
if (action.status !== "pending") return c.json({ error: ... }, 400);
// sem comparar action.requested_by com c.get("user").email
const reviewer = c.get("user").email;
await c.env.DB.prepare(`UPDATE google_ads_actions SET status='approved', reviewed_by=? ...`)
});
```

Condições de explorabilidade: Sempre explorável por qualquer usuário autenticado — basta chamar propose-* e depois approve com a própria sessão. Mitigado em parte por F1 já afetar o mesmo grupo de usuários (equipe DDLab), mas é um controle de processo que a própria feature (uma fila de aprovação) sugere ter a intenção de prover e hoje não provê.

ALTA

F3 · Endpoint POST /api/refresh dispara custo real (BigQuery) sem limite de taxa; a proteção REFRESH_TOKEN documentada nunca é verificada no código

`worker/index.ts:50-64, .dev.vars.example:15-16`

```
app.post("/api/refresh", async (c) => {
const account = c.req.query("account") || undefined;
// ... nenhuma verificação de c.env.REFRESH_TOKEN
c.executionCtx.waitUntil(runEtl(c.env, { accountIds: ..., lookbackDays: ..., facts }));
return c.json({ started: true, ... });
});
```

Condições de explorabilidade: Explorável por qualquer usuário autenticado: chamadas repetidas a /api/refresh?days=400 para todas as contas geram consultas pesadas e repetidas ao BigQuery (a doc do próprio README/DEPLOY recomenda backfill de 400 dias), inflando a fatura do GCP sem limite. O mesmo vale para `?facts=term_classification`/`keyword_volume`, que consomem quota da API do Google Ads e da API da Anthropic/Gemini/DeepSeek (o próprio README alerta sobre isso).

BAIXA

F4 · GET /api/introspect expõe o schema interno do BigQuery (nomes de coluna/tabela) para qualquer usuário autenticado

`worker/index.ts:23-39`

```
app.get("/api/introspect", async (c) => {
const table = c.req.query("table");
if (!table || !/^[a-zA-Z0-9_]+$/.test(table)) return c.json({ error: "?table inválido" }, 400);
...
});
```

Condições de explorabilidade: Baixo risco — exige já estar autenticado via Access; vaza apenas nomes/tipos de coluna do dataset do BigQuery, não dados de linha.

Recomendações priorizadas

Prio.	Recomendação	Sev.
P1	Implementar de fato a checagem de REFRESH_TOKEN e/ou rate limiting em POST /api/refresh (F3) — agora cobre também quota de Google Ads API e Anthropic/Gemini/DeepSeek.	ALTA
P1	Decidir e documentar o modelo de autorização (uso interno total-acesso vs. RBAC por conta) e, no mínimo, priorizar checagem de posse nos endpoints de /ads-actions, que movem dinheiro real de clientes (F1).	ALTA
P2	Adicionar segregação entre quem propõe e quem aprova uma mutação no Google Ads, ou documentar que a auto-aprovação é aceita pela agência (F2).	MÉDIA
P3	Restringir /api/introspect a ambiente de desenvolvimento/staging (F4).	BAIXA
P3	Adicionar teste automatizado que falhe o build/deploy se CF_ACCESS_TEAM_DOMAIN/AUD estiverem vazios em modo produção (consolidar a correção do antigo F1).	INFORMATIVA

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue no repositório.

--- ISSUE 1 ---

[Segurança] Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e escrita a todas as contas de clientes, incluindo mutações reais no Google Ads

````
[Segurança] Ausência total de RBAC — qualquer usuário autenticado tem acesso irrestrito de leitura e
escrita a todas as contas de clientes, incluindo mutações reais no Google Ads

**Labels sugeridas:** security, severidade:alta

## Descrição do problema
Não existe conceito de papel (admin/editor/leitor) nem de dono de conta em nenhuma camada do backend
(confirmado: nenhuma ocorrência de role/isAdmin/canEdit em worker/ ou src/). accessMiddleware() só
verifica "é um e-mail permitido pela policy do Cloudflare Access" (DEPLOY.md linha 91: `@ddlab.com.br` ou
e-mails individuais) — a partir daí, o mesmo usuário pode ler dados de todas as contas, editar
metas/orçamento/briefing (POST /api/settings/account), disparar ETL pago para qualquer conta (POST
/api/refresh), gerar análises de IA pagas (POST /api/analysis, /api/diagnostico) e — desde a integração de
escrita com o Google Ads (Fases 1 e 2) — propor E aprovar mutações reais em qualquer conta: negativar
termos, pausar/reativar campanhas e palavras-chave (worker/routes/adsActions.ts). Não há checagem de posse
— é tudo-ou-nada.

## Por que é explorável
Sempre explorável por qualquer conta liberada na policy do Cloudflare Access. Era um risco de exposição de
dados/custo na 1ª rodada da auditoria; com a integração de escrita do Google Ads (commits #36-#38) o mesmo
modelo tudo-ou-nada agora também cobre ações com efeito direto em campanhas reais de clientes (pausar
campanha, negativar termo), o que amplia o dano possível de um acesso indevido ou de um colaborador
despreparado. Ainda não documentado como decisão de produto deliberada em lugar nenhum do repositório.

## Evidência
`worker/routes/reports.ts:753-803, worker/routes/adsActions.ts:60-261, worker/routes/analysis.ts:375-511,
worker/index.ts:50-64`

```
reports.post("/settings/account", async (c) => {
const body = await c.req.json<{ id: string; ... }>();
if (!body.id) return c.json({ error: "id obrigatório" }, 400);
await c.env.DB.prepare(`UPDATE dim_account SET ... WHERE id = ?`)...
// nenhuma verificação de quem pode editar a conta `body.id`
});

adsActions.post("/ads-actions/:id/approve", async (c) => {
// qualquer usuário autenticado aprova a mutação de QUALQUER conta
await addCampaignNegativeKeywords(c.env, payload.customerId, ...);
});
```

## Impacto
Qualquer pessoa com acesso à ferramenta pode alterar configurações financeiras de qualquer cliente, pausar
campanhas ou negativar termos na conta de Google Ads real de qualquer cliente, disparar custos reais
(BigQuery, Anthropic, Google Ads API) em qualquer conta, e ver o briefing confidencial de todos os
clientes da agência — não apenas dos que gerencia.

## Sugestão de correção
Definir explicitamente o modelo de autorização: se for intencional (equipe interna enxuta), documentar a
decisão formalmente (ex.: no README/DEPLOY.md); se não, adicionar uma tabela de vínculo usuário↔conta (ou
papel) e checar posse em cada endpoint de escrita — com prioridade para os endpoints de /ads-actions, que
movem dinheiro de clientes de verdade.

## Critérios de aceite
- [ ] Comportamento corrigido conforme a sugestão de correção acima
- [ ] Cenário de exploração descrito não é mais reproduzível
- [ ] Teste (automatizado ou checklist manual) cobrindo a regressão adicionado
- [ ] Revisão de código por outra pessoa antes do merge
````

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Fluxo de aprovação de mutações no Google Ads não exige segregação entre quem propõe e quem aprova (sem maker-checker)

```
[Segurança] Fluxo de aprovação de mutações no Google Ads não exige segregação entre quem propõe e quem
aprova (sem maker-checker)

**Labels sugeridas:** security, severidade:media

## Descrição do problema
POST /api/ads-actions/:id/approve só verifica se a ação ainda está `pending` — não compara
`action.requested_by` com `c.get("user").email`. O mesmo usuário que cria a proposta (negativar termo,
pausar campanha/palavra-chave) pode aprová-la sozinho e disparar a mutação real no Google Ads
(`addCampaignNegativeKeywords`/`setCampaignStatus`/`setKeywordStatus` em worker/googleAds.ts) sem qualquer
segunda pessoa revisar. A fila de aprovação existe para dar visibilidade humana antes da mutação, mas nada
no backend impede que essa revisão seja feita pela mesma pessoa que propôs.

## Por que é explorável
Sempre explorável por qualquer usuário autenticado — basta chamar propose-* e depois approve com a própria
sessão. Mitigado em parte por F1 já afetar o mesmo grupo de usuários (equipe DDLab), mas é um controle de
processo que a própria feature (uma fila de aprovação) sugere ter a intenção de prover e hoje não provê.

## Evidência
`worker/routes/adsActions.ts:197-245`

```
adsActions.post("/ads-actions/:id/approve", async (c) => {
const action = await loadPending(c.env, id);
if (action.status !== "pending") return c.json({ error: ... }, 400);
// sem comparar action.requested_by com c.get("user").email
const reviewer = c.get("user").email;
await c.env.DB.prepare(`UPDATE google_ads_actions SET status='approved', reviewed_by=? ...`)
});
```

Impacto
Erro humano ou ação indevida de um único colaborador pode pausar uma campanha ou negativar um termo
relevante em produção, no Google Ads real do cliente, sem nenhuma segunda revisão — o que é justamente o
cenário que a fila de aprovação foi desenhada para evitar.

Sugestão de correção
Registrar `requested_by` e comparar com o usuário autenticado em /approve; se forem iguais, exigir uma
segunda pessoa (retornar 403) ou, no mínimo, sinalizar claramente no frontend/log que a ação foi
auto-aprovada, para decisão consciente da agência sobre se isso é aceitável.

Critérios de aceite
- [ ] Comportamento corrigido conforme a sugestão de correção acima
- [ ] Cenário de exploração descrito não é mais reproduzível
- [ ] Teste (automatizado ou checklist manual) cobrindo a regressão adicionado
- [ ] Revisão de código por outra pessoa antes do merge
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Rotas administrativas/de debug sem controles adicionais (refresh sem rate limit, introspect exposto)

```
[Segurança] Rotas administrativas/de debug sem controles adicionais

**Labels sugeridas:** security, severidade:alta


---
### Endpoint POST /api/refresh dispara custo real (BigQuery) sem limite de taxa; a proteção REFRESH_TOKEN
documentada nunca é verificada no código (worker/index.ts:50-64, .dev.vars.example:15-16)

**Descrição:** .dev.vars.example documenta REFRESH_TOKEN como "Token simples para proteger POST
/api/refresh quando chamado fora do Access", e Env.REFRESH_TOKEN existe em worker/env.ts:19. Porém nenhum
arquivo do worker lê ou compara REFRESH_TOKEN — o handler `app.post("/api/refresh", ...)` em
worker/index.ts:50 dispara `runEtl()` (que consulta o BigQuery, custo real por bytes processados) sem
checar esse token nem qualquer outro controle além do accessMiddleware() genérico, e sem rate limiting.
Sem mudança desde a 1ª rodada da auditoria.
```

```
**Por que é explorável:** Explorável por qualquer usuário autenticado: chamadas repetidas a /api/refresh?days=400 para todas as contas geram consultas pesadas e repetidas ao BigQuery (a doc do próprio README/DEPLOY recomenda backfill de 400 dias), inflando a fatura do GCP sem limite. O mesmo vale para `?facts=term_classification`/`keyword_volume`, que consomem quota da API do Google Ads e da API da Anthropic/Gemini/DeepSeek (o próprio README alerta sobre isso).

**Evidência:**
```
app.post("/api/refresh", async (c) => {
const account = c.req.query("account") || undefined;
// ... nenhuma verificação de c.env.REFRESH_TOKEN
c.executionCtx.waitUntil(runEtl(c.env, { accountIds: ..., lookbackDays: ..., facts }));
return c.json({ started: true, ... });
});
```

**Impacto:** Consumo de custo (billing do GCP/BigQuery/Anthropic/Google Ads API) fora de controle, possível esgotamento de cota.

**Sugestão de correção:** Implementar de fato a checagem de REFRESH_TOKEN (ou removê-la da documentação se não for usada) e adicionar rate limiting/debounce por conta (ex.: recusar novo refresh se um já rodou nos últimos N minutos, usando meta_refresh.last_run_at).


---
### GET /api/introspect expõe o schema interno do BigQuery (nomes de coluna/tabela) para qualquer usuário autenticado (worker/index.ts:23-39)

**Descrição:** O endpoint de depuração /api/introspect roda `SELECT column_name, data_type FROM INFORMATION_SCHEMA.COLUMNS` para a tabela informada via querystring e devolve o schema. Está corretamente sob accessMiddleware() e o parâmetro `table` é validado por regex (`^[a-zA-Z0-9_]+$`) antes de entrar na query (sem SQL injection), mas é uma rota de debug interno deixada acessível em produção para qualquer usuário do Access, não só para quem ajusta o ETL. Sem mudança desde a 1ª rodada da auditoria.

**Por que é explorável:** Baixo risco — exige já estar autenticado via Access; vaza apenas nomes/tipos de coluna do dataset do BigQuery, não dados de linha.

**Evidência:**
```
app.get("/api/introspect", async (c) => {
const table = c.req.query("table");
if (!table || !/^[a-zA-Z0-9_]+$/.test(table)) return c.json({ error: "?table inválido" }, 400);
...
});
```

**Impacto:** Reconhecimento de schema interno por um usuário autenticado que não deveria ter esse nível de acesso operacional.

**Sugestão de correção:** Mover para uma rota administrativa separada, ou gate-la atrás de uma flag de ambiente (só disponível em dev/staging).


## Critérios de aceite
- [ ] REFRESH_TOKEN validado em POST /api/refresh (ou removido da documentação se descontinuado)
- [ ] Rate limiting/debounce por conta implementado em POST /api/refresh
- [ ] /api/introspect restrito a ambiente não-produção
- [ ] Teste cobrindo a regressão adicionado
```

--- FIM ISSUE 3 ---